

Proj3: 事务管理与并发控制

1 实验目的

本实验基于 Project1 中已经完成的 Olist 电商数据库，构造一个商品库存抢购场景，用多线程模拟多个用户同时访问同一条库存记录。在无锁模式下，我通过普通 `SELECT` 和应用层延迟复现了数据库并发操作中的“丢失修改”异常；在加锁模式下，我使用 `SELECT ... FOR UPDATE` 对目标元组加排他锁，使读、改、写过程在事务内部被串行化，从而保证最终库存结果正确。

通过本次实验，可以更直观地理解事务的隔离性、并发调度中的写覆盖问题，以及 openGauss 中悲观锁对共享资源访问的控制作用。

2 实验环境

项目	配置
数据库	openGauss
数据库名	ecommerce_db
Schema	proj1_3nf
编程语言	Python
主要库	threading、psycpg2、pandas、streamlit
前端框架	Streamlit
运行入口	project3_app.py

3 业务场景定义

Project1 的主题是 Olist 电商交易数据，原始三范式结构中包含 `products`、`orders`、`order_items`、`order_reviews` 等实体，但并没有库存字段。为了满足本实验限量资源争夺的要求，我在 `proj1_3nf` 中新增了实验表 `product_inventory`，用于表示某个商品的库存。

核心场景为：多个用户同时抢购同一个商品，每个用户成功抢购后，目标商品库存减少 1 件。

实验表结构如下：

```
1 CREATE TABLE IF NOT EXISTS product_inventory (  
2     product_id VARCHAR(32) PRIMARY KEY,  
3     stock_count INTEGER NOT NULL CHECK (stock_count >= 0),  
4     updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
5     CONSTRAINT fk_product_inventory_product  
6         FOREIGN KEY (product_id)  
7         REFERENCES products(product_id)  
8 );
```

表中各字段含义如下：

- `product_id`：商品编号，引用 Project1 中的 `products(product_id)`。
- `stock_count`：当前商品库存，是本实验中的共享可变资源。
- `updated_at`：库存记录最后更新时间，用于观察数据变化。

这里没有直接修改 Project1 的 `products` 表，而是选择新增一个实验表，这样既能保持 Project1 的规范化数据结构不被破坏，也可以围绕真实商品编号构造并发抢购场景。`product_inventory` 和 `products` 通过外键关联，保证实验库存必须属于已有商品。

初始化脚本为 `project3_setup.sql`，执行后会为若干商品初始化库存。Streamlit 应用在每次实验开始前也会将当前选中商品的库存重置为指定初始值，从而保证实验结果可重复。

4 代码整体框架

本实验的 Python 程序主要分为三个模块：

1. 数据库连接与实验表准备。
2. 并发事务处理逻辑。
3. Streamlit 可视化实验控制台。

4.1 数据库连接与事务配置

```
1 def get_conn():
2     """创建数据库连接，并进入 proj1 的 3NF schema。"""
3     conn = psycopg2.connect(**DB_CONFIG)
4     conn.set_session(
5         isolation_level=extensions.ISOLATION_LEVEL_READ_COMMITTED,
6         autocommit=False,
7     )
8     with conn.cursor() as cur:
9         cur.execute(f"SET search_path TO {SCHEMA}, public;")
10    conn.commit()
11    return conn
```

该函数负责创建数据库连接，并将事务隔离级别设置为 `Read Committed`。这里显式设置 `autocommit=False`，是为了保证每个线程的读取、等待、更新和提交都处在同一个事务过程中。对于加锁模式，`SELECT ... FOR UPDATE` 获取到的行锁会一直保持到事务提交或回滚。

4.2 实验表初始化

```
1 def ensure_inventory_table():
2     """确保 Project3 使用的商品库存实验表存在。"""
3     sql = f"""
4         CREATE TABLE IF NOT EXISTS {INVENTORY_TABLE} (
5             product_id VARCHAR(32) PRIMARY KEY,
6             stock_count INTEGER NOT NULL CHECK (stock_count >= 0),
7             updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
8             CONSTRAINT fk_product_inventory_product
9                 FOREIGN KEY (product_id)
10                REFERENCES products(product_id)
11        );
12    """
```

该模块实现了实验表的自动检查和创建。即使只运行 Streamlit 应用而没有手动执行初始化 SQL，只要 Project1 的 `products` 表存在，应用也能自动准备 `product_inventory` 表。

4.3 库存重置

```
1 def init_inventory(product_id, initial_count):
2     """重置目标商品库存，确保每次实验可重复。"""
3     insert_sql = f"""
4         INSERT INTO {INVENTORY_TABLE} (product_id, stock_count)
5         SELECT %s, %s
6         WHERE EXISTS (
7             SELECT 1 FROM products WHERE product_id = %s
8         )
9         AND NOT EXISTS (
10            SELECT 1 FROM {INVENTORY_TABLE} WHERE product_id = %s
11        );
12    """
13     update_sql = f"""
14         UPDATE {INVENTORY_TABLE}
15         SET stock_count = %s,
16             updated_at = CURRENT_TIMESTAMP
17         WHERE product_id = %s;
18    """
```

每次点击测试按钮前，程序都会调用 `init_inventory` 将目标商品库存重置为侧边栏中设置的初始库存。这样做可以避免前一次实验结果影响下一次实验。

5 并发事务处理逻辑

核心函数为 `concurrency_worker`。每个线程代表一个并发用户，在线程中独立创建数据库连接，并执行一次抢购事务。

5.1 无锁读取路径

```
1 select_sql = f"""
2     SELECT stock_count
3     FROM {INVENTORY_TABLE}
4     WHERE product_id = %s;
5 """
6
7 cur.execute(select_sql, (product_id,))
8 current_stock = row[0]
9 time.sleep(delay_time)
10 new_stock = current_stock - 1
11 cur.execute(update_sql, (new_stock, product_id))
12 conn.commit()
```

无锁模式下，线程只是普通读取当前库存，不会阻塞其他线程。由于读取之后加入了 `time.sleep(delay_time)`，多个线程会在同一个时间窗口内读到相同的旧值。

例如初始库存为 100 时，10 个线程可能都读取到 100。每个线程随后都计算出 99，并执行 `UPDATE` 写回数据库。最后数据库库存只从 100 变成 99，而不是预期的 90。

分析：这是典型的丢失修改。多个事务都基于同一个旧值进行计算，后提交的事务覆盖了前面事务已经写入的结果。从最终数据库状态看，部分已经发生过的扣减操作被覆盖掉了。

5.2 加锁读取路径

```
1 select_sql = f"""
2     SELECT stock_count
3     FROM {INVENTORY_TABLE}
4     WHERE product_id = %s
5     FOR UPDATE;
6 """
```

加锁模式只改变读取语句，在 `SELECT` 后加入 `FOR UPDATE`。该语句会对查询到的目标库存行加排他锁。一个线程拿到锁后，其他线程必须等待该事务提交或回滚，才能继续读取同一行。

因此线程执行过程变为：

1. 线程 1 读取库存 100，加锁并写回 99。
2. 线程 2 等待线程 1 提交后，读取最新库存 99，并写回 98。
3. 后续线程依次读取 98、97、96 等最新值。
4. 10 个线程全部执行后，库存准确变为 90。

分析：`FOR UPDATE` 并不是简单地让 `UPDATE` 更安全，而是将读操作本身也纳入锁控制，使读、改、写过程成为一个受保护的临界区。这对应本实验希望观察的悲观锁机制。

5.3 异常处理与回滚

```
1 except Exception as e:
2     if conn:
3         conn.rollback()
4         append_log(log_list, log_lock, f"[{thread_name}] 数据库异常，事务回滚: {e}")
5 finally:
6     if conn:
7         conn.close()
```

每个 worker 都包含异常捕获逻辑。当数据库连接、SQL 执行或事务提交出现异常时，程序会执行 `rollback` 并记录日志，避免事务处于未提交状态。

6 可视化交互设计

本实验使用 Streamlit 构建高并发测试控制台。界面主要包含以下功能：

- 目标商品选择：从 `products` 和 `product_inventory` 中读取商品编号、类别和当前库存。
- 实验参数设置：配置初始库存、并发线程数和业务处理延迟。
- 无锁测试按钮：启动普通 `SELECT` 模式，用于复现丢失修改。
- 加锁测试按钮：启动 `SELECT ... FOR UPDATE` 模式，用于验证并发控制。
- 实时库存显示：通过 `st.metric` 展示数据库中的实际库存。
- 事务日志展示：使用 `st.text_area` 输出每个线程的读取值、写回值和异常信息。
- 结果判定：自动比较预期库存和实际库存，并给出实验结论。

控制台中的核心结果判断如下：

```
1 expected_sold = min(initial_stock, thread_count)
2 expected_stock = initial_stock - expected_sold
3 actual_sold = initial_stock - final_stock
```

其中：

- `expected_sold`：理论应售出的商品数量。
- `expected_stock`：并发执行后理论剩余库存。
- `actual_sold`：数据库中实际减少的库存数量。

如果无锁模式下 `final_stock > expected_stock`，说明库存减少得比理论值少，程序判定发生丢失修改；如果加锁模式下 `final_stock == expected_stock`，说明排他锁控制有效，结果符合预期。

7 实验过程与结果分析

7.1 数据库前置检查

执行 Project1 共享检查脚本后，`proj1_3nf` 中核心表均有数据，说明商品库存实验可以基于真实商品编号展开。

`product_inventory` 初始化后，部分实验库存记录如下：

product_id	product_category_name	stock_count
00066f42aeeb9f3007548bb9d3f33c38	perfumaria	100
00088930e925c41fd95ebfe695fd2655	automotivo	100
0009406fd7479715e4bef61dd91f2462	cama_mesa_banho	100
000b8f95fcb9e0096488278317764d19	utilidades_domesticas	100
000d9be29b5207b54e86aa1b1ac54872	relogios_presentes	100

本次验证选择商品：

```
1 product_id = 00066f42aeeb9f3007548bb9d3f33c38
```

测试参数为：

参数	值
初始库存	100
并发线程数	10
业务处理延迟	0.2 秒

7.2 无锁并发测试

无锁模式下，10 个线程同时执行普通 `SELECT`，读取到的库存如下：

```
1 unlocked_reads = [100, 100, 100, 100, 100, 100, 100, 100, 100, 100]
2 unlocked_writes = [99, 99, 99, 99, 99, 99, 99, 99, 99, 99]
```

最终结果为：

指标	结果
预期剩余库存	90
实际剩余库存	99
理论应售出	10
实际售出	1
是否观测到丢失修改	是

场景 A：无锁并发测试

普通 SELECT 读取旧库存，休眠后按旧值写回，用于复现丢失修改。

启动无锁测试

已将商品 00088930e925c41fd95ebfe695fd2655 库存重置为 100

数据库实际库存

98 件

理论应售出

10 件

实际售出

2 件

预期剩余库存

90 件

事务执行日志

17:27:56.441 [User-4] 无锁读取库存=100

17:27:56.516 [User-8] 无锁读取库存=100

17:27:56.527 [User-3] 无锁读取库存=100

17:27:56.541 [User-2] 无锁读取库存=100

17:27:56.542 [User-7] 无锁读取库存=100

17:27:56.543 [User-1] 无锁读取库存=100

17:27:56.557 [User-4] 抢购成功，写回库存=99

17:27:56.572 [User-5] 无锁读取库存=99

17:27:56.572 [User-6] 无锁读取库存=99

17:27:56.580 [User-9] 无锁读取库存=99

17:27:56.582 [User-10] 无锁读取库存=99

17:27:56.626 [User-8] 抢购成功，写回库存=99

17:27:56.635 [User-3] 抢购成功，写回库存=99

17:27:56.650 [User-2] 抢购成功，写回库存=99

17:27:56.656 [User-7] 抢购成功，写回库存=99

17:27:56.661 [User-1] 抢购成功，写回库存=99

17:27:56.680 [User-5] 抢购成功，写回库存=98

17:27:56.686 [User-6] 抢购成功，写回库存=98

17:27:56.693 [User-10] 抢购成功，写回库存=98

17:27:56.699 [User-9] 抢购成功，写回库存=98

观测到丢失修改：预期库存应为 90，实际库存为 98，说明部分线程基于旧库存写回并覆盖了其他事务结果。

分析：从日志可以看出，所有线程都读取到了库存 100，并在休眠后写回 99。虽然 10 个事务都执行了扣减逻辑，但这些扣减都是基于同一个旧值计算出的，因此后续事务写回时覆盖了前面事务的修改。最终库存只减少 1 件，和预期减少 10 件不一致，符合丢失修改异常的特征。

7.3 加锁并发测试

加锁模式下，10 个线程执行 `SELECT ... FOR UPDATE`，读取和写回过程如下：

- 1 | locked_reads = [100, 99, 98, 97, 96, 95, 94, 93, 92, 91]
- 2 | locked_writes = [99, 98, 97, 96, 95, 94, 93, 92, 91, 90]

最终结果为：

指标	结果
预期剩余库存	90
实际剩余库存	90
理论应售出	10
实际售出	10
加锁结果是否正确	是

场景 B：排他锁并发测试

使用 SELECT ... FOR UPDATE 锁定库存行，验证事务串行化后的正确结果。

启动加锁测试

已将商品 00088930e925c41fd95ebfe695fd2655 库存重置为 100

数据库实际库存

90 件

理论应售出

10 件

实际售出

10 件

预期剩余库存

90 件

事务执行日志

17:30:59.711 [User-2] 等待获取商品行排他锁

17:30:59.714 [User-2] 加锁读取库存=100

17:30:59.719 [User-7] 等待获取商品行排他锁

17:30:59.738 [User-8] 等待获取商品行排他锁

17:30:59.749 [User-6] 等待获取商品行排他锁

17:30:59.758 [User-10] 等待获取商品行排他锁

17:30:59.766 [User-3] 等待获取商品行排他锁

17:30:59.777 [User-5] 等待获取商品行排他锁

17:30:59.789 [User-9] 等待获取商品行排他锁

17:30:59.800 [User-1] 等待获取商品行排他锁

17:30:59.809 [User-4] 等待获取商品行排他锁

17:30:59.822 [User-2] 抢购成功，写回库存=99

17:30:59.822 [User-7] 加锁读取库存=99

17:30:59.930 [User-7] 抢购成功，写回库存=98

17:30:59.930 [User-1] 加锁读取库存=98

17:31:00.038 [User-8] 加锁读取库存=97

17:31:00.039 [User-1] 抢购成功，写回库存=97

17:31:00.145 [User-9] 加锁读取库存=96

17:31:00.145 [User-8] 抢购成功，写回库存=96

17:31:00.253 [User-9] 抢购成功，写回库存=95

17:31:00.253 [User-4] 加锁读取库存=95

17:31:00.362 [User-4] 抢购成功，写回库存=94

17:31:00.362 [User-5] 加锁读取库存=94

17:31:00.477 [User-5] 抢购成功，写回库存=93

17:31:00.477 [User-3] 加锁读取库存=93

17:31:00.584 [User-3] 抢购成功，写回库存=92

17:31:00.584 [User-10] 加锁读取库存=92

17:31:00.697 [User-10] 抢购成功，写回库存=91

17:31:00.697 [User-6] 加锁读取库存=91

17:31:00.808 [User-6] 抢购成功，写回库存=90

加锁测试通过：automotivo 商品库存从 100 正确扣减到 90。

分析：加锁后，每个线程读取到的库存都是前一个线程提交后的最新值。库存变化呈现 100、99、98、97 这样的连续递减过程，最终准确到达 90。由此可见，`SELECT ... FOR UPDATE` 对目标元组施加了有效的排他锁，使并发事务在访问同一库存记录时被串行化。

8 与 ACID 特性的关系

本实验重点体现了事务隔离性和一致性之间的关系。

在无锁模式下，每个事务内部看似都完成了读取、计算、更新、提交的完整流程，但事务之间缺少对共享记录的互斥控制，导致多个事务读取同一个旧值并覆盖写回。单个事务的原子性并不能自动避免并发覆盖问题。

在加锁模式下，`FOR UPDATE` 将目标库存行作为临界资源保护起来。事务在持有锁期间完成读、改、写，其他事务必须等待，因此数据库可以保证每个事务都基于最新库存继续执行。这样既保持了事务内部操作的原子性，也通过隔离性维护了最终库存的一致性。悲观锁的作用是在冲突可能发生前主动限制并发访问，从而避免写覆盖。

9 实验总结

本实验基于 Project1 的 Olist 电商数据库，新增商品库存表并构建了多线程抢购场景。通过无锁模式，我观察到多个线程同时读取同一库存值并写回相同结果，最终库存只减少 1 件，成功复现了丢失修改异常。通过加锁模式，我使用 `SELECT ... FOR UPDATE` 对库存行加排他锁，使线程依次读取最新库存并完成扣减，最终库存从 100 正确变为 90。

通过本次实验，我对事务管理中的隔离性有了更具体的理解。并发问题并不一定表现为 SQL 执行失败，更多时候是程序可以正常运行，但最终数据状态不符合业务语义。`FOR UPDATE` 的意义在于把读操作也纳入锁控制，保证读、改、写过程不会被其他事务穿插破坏。

总体而言，本实验将数据库理论中的丢失修改、事务隔离和排他锁机制转化为可以观察的电商库存场景，使我更清楚地认识到并发控制对于数据库一致性的实际价值。